

Meghan Gillen
May 7, 2026

Lab 6 - EE 446

Section 1: Edge Impulse Results

Public Edge Impulse project link: <https://studio.edgeimpulse.com/public/989441/live>

Part 1 Questions:

1. Raw inputs: State the raw inputs used in the Edge Impulse project.

The raw inputs were accelerometer readings along the X, Y, and Z axes (accX, accY, and accZ). These are readings probably from a sensor placed on the fan that will feel it moving.

2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.

The type of features passed in are the spectral features. There are 33 input features. The names of 5 actual feature names are RMS, Peak 1 Height, Spectral Power 2.0 - 5.0, Peak 1 Freq, and Peak 2 Freq. Each of these features gets passed in per their dimension in 3D (X, Y, Z).

3. NN classifier outputs: State the outputs of the NN classifier.

The outputs of the NN classifier are features labelled with the label either "nominal" or "off". It says output has 2 features (one of each of those labels).

4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.

The anomaly detector is passed in the spectral feature as well. It outputs an anomaly score. The feature names used to train the anomaly detector are accX RMS, accX Peak 1 Height, accY Spectral Power 2.0-5.0 and accZ Spectral power 2.0 -5.0. These are selected out of all features to be included in the model training.

5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.

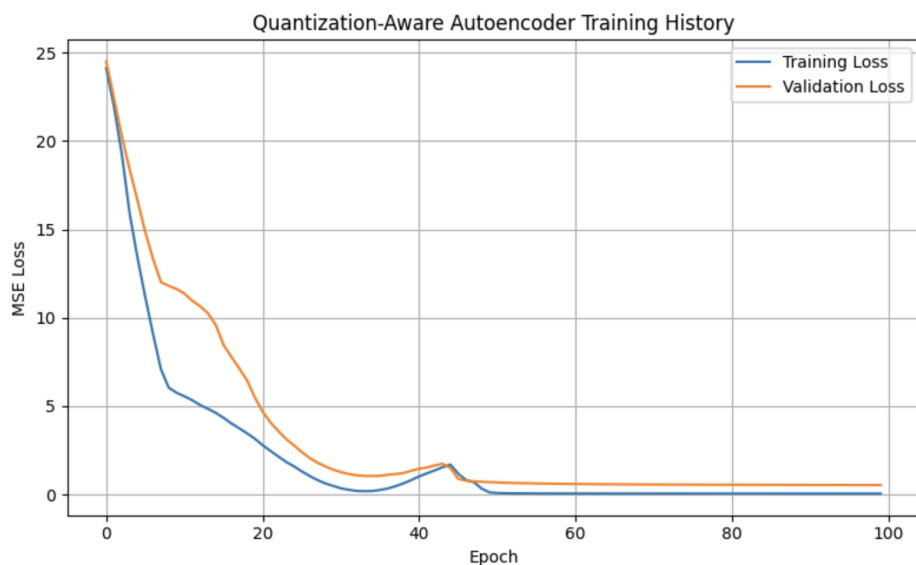
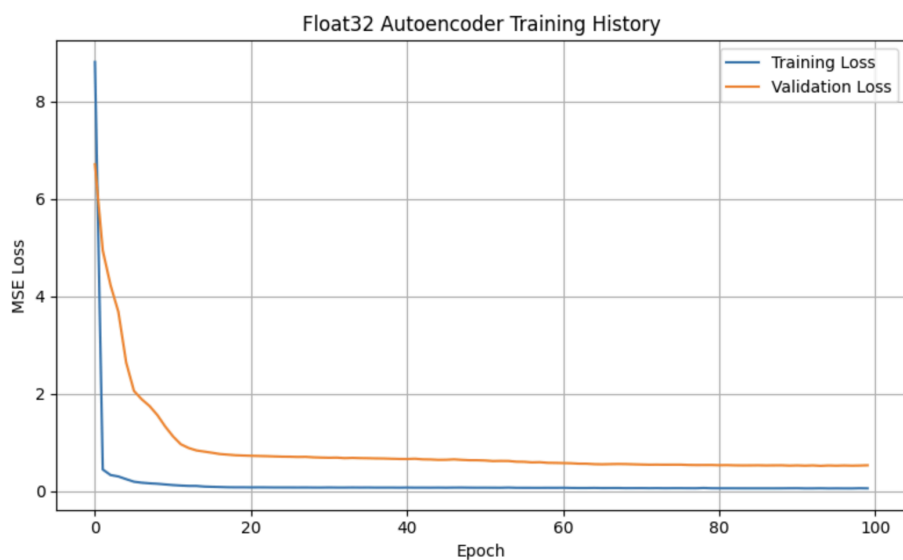
```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 584 us, anomaly 552 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.234375
  off: 0.765625
Anomaly prediction: -0.408210
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 613 us, anomaly 538 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.062500
  off: 0.937500
Anomaly prediction: 1000.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 40 ms, inference 6 ms, anomaly 514 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.968750
  off: 0.031250
Anomaly prediction: 77.560997
Starting inferencing in 2 seconds...
Sampling...
```

6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

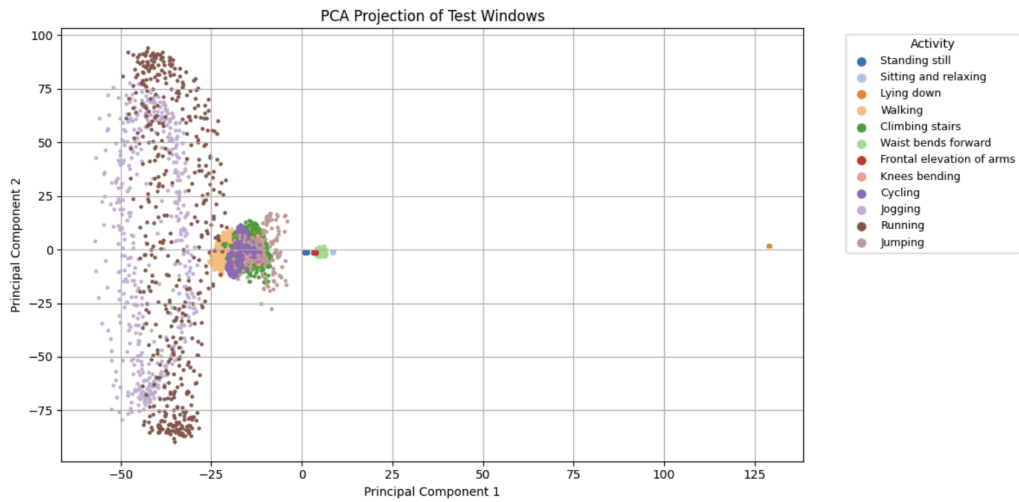
When I moved the board in a circle, trying to simulate a fan, the error went to 1000.00, which I want to assume is the maximum anomaly score. When it was completely flat, laying on the table, and still, the value was around 0, but often negative. In the photo above, it shows when it was laying on the table (-0.408), when I was spinning it (1000.0) and finally when I had just put it down (77.56). I don't think I would always trust it since there seems to be a 'max' to the anomaly score. However, it maxes out when it is being spun which is the condition of a fan so I think currently it is not a great classifier, especially since OFF is also higher when it is being moved, which is wrong.

Section 2: Autoencoder Model Results

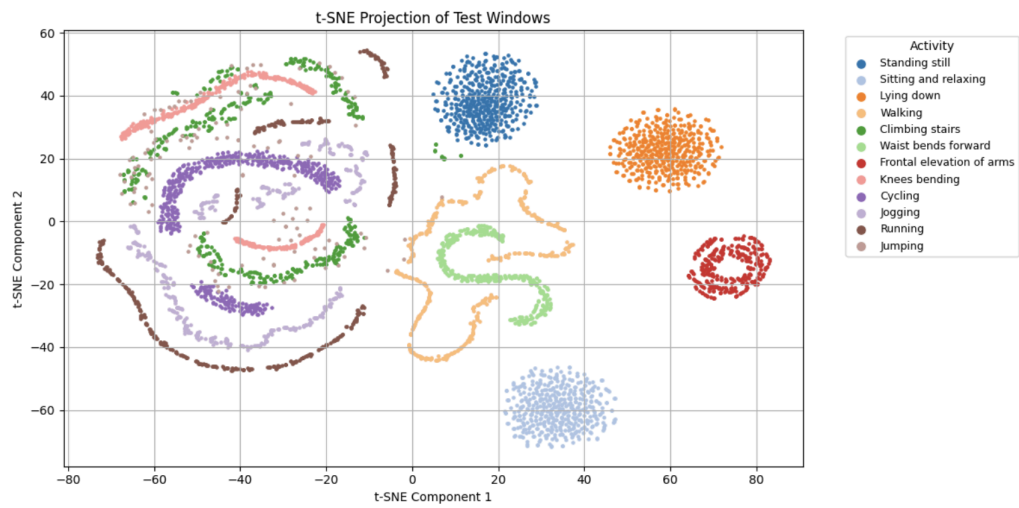
Training and validation loss curves



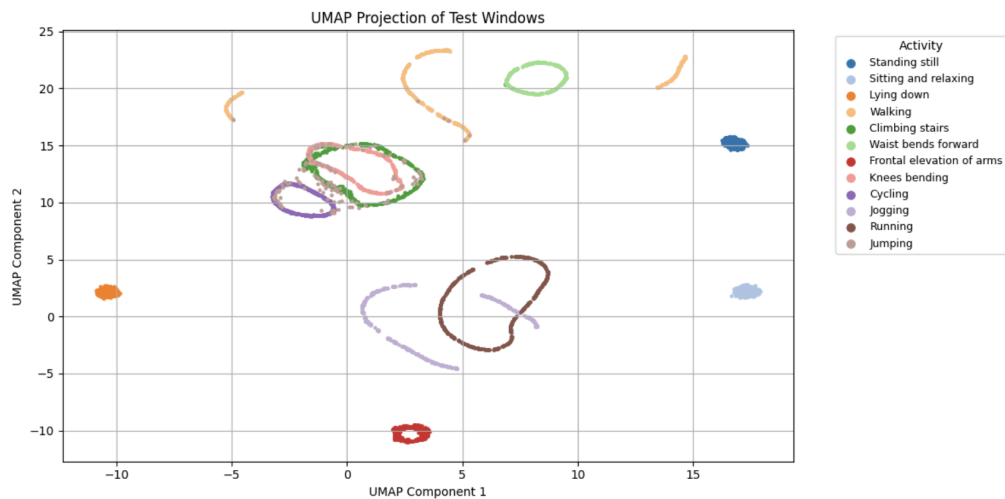
PCA visualization



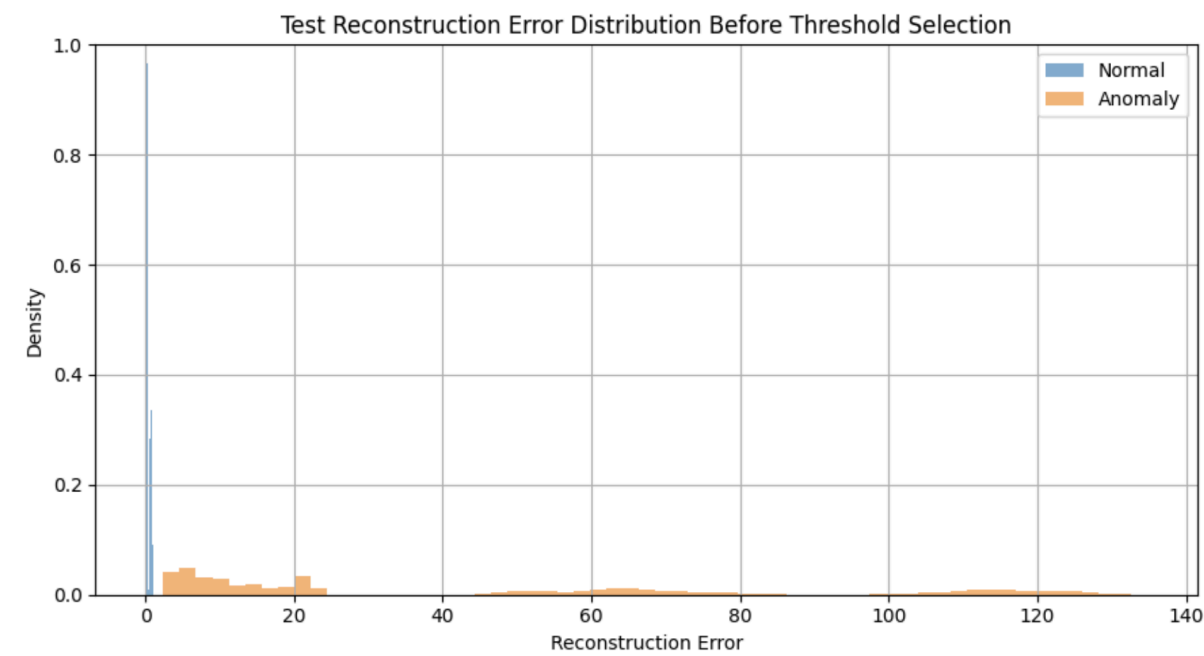
t-SNE visualization



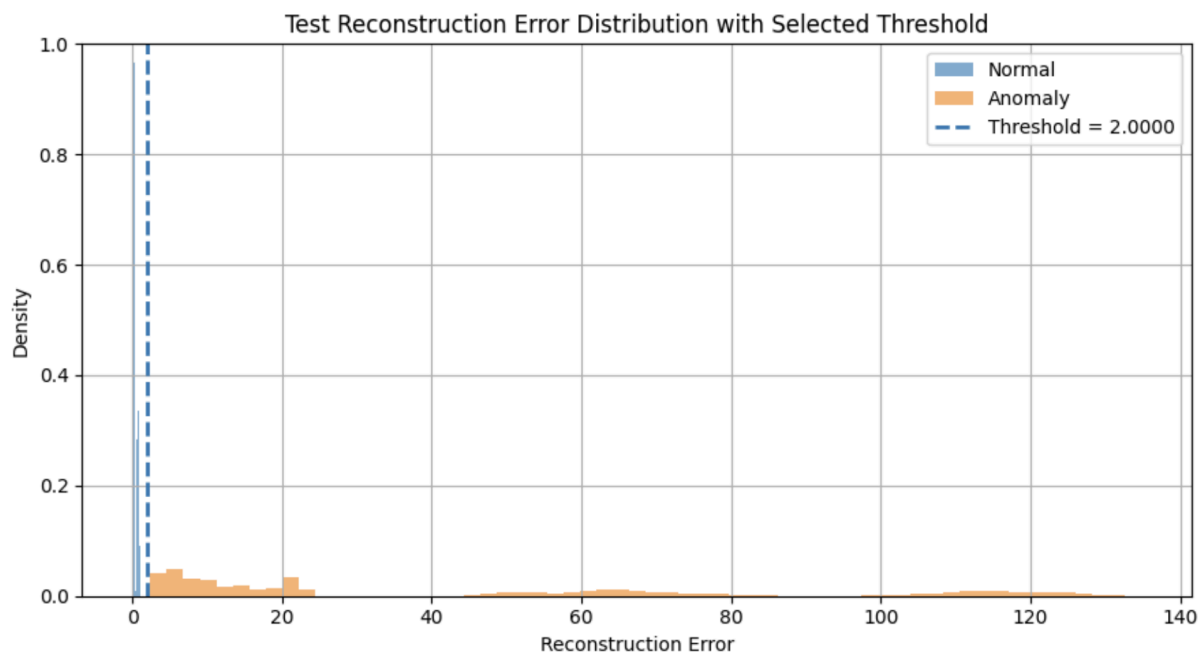
UMAP visualization



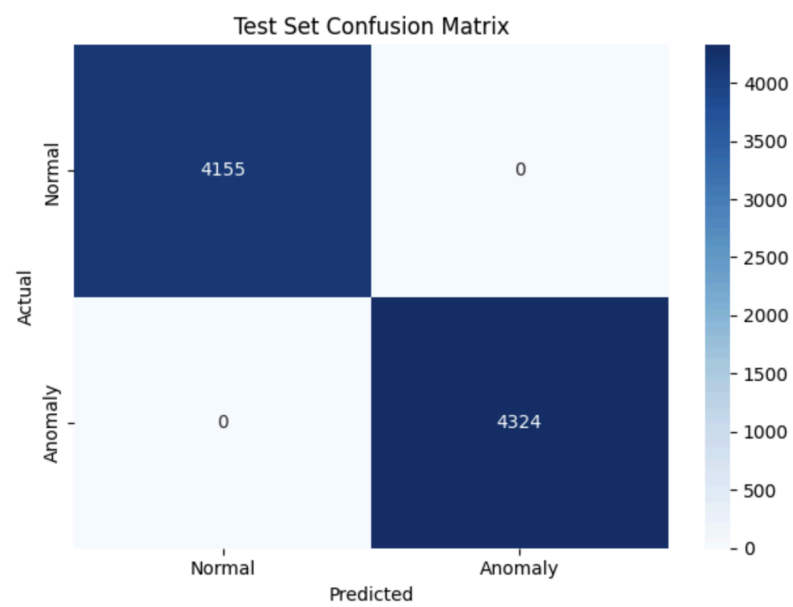
Reconstruction error distribution



Selected anomaly threshold



Confusion matrix



Classification report or evaluation metrics

Train set evaluation				
	precision	recall	f1-score	support
Normal (0)	1.00	1.00	1.00	10478
Anomaly (1)	1.00	1.00	1.00	10872
accuracy			1.00	21350
macro avg	1.00	1.00	1.00	21350
weighted avg	1.00	1.00	1.00	21350
Test set evaluation				
	precision	recall	f1-score	support
Normal (0)	1.00	1.00	1.00	4155
Anomaly (1)	1.00	1.00	1.00	4324
accuracy			1.00	8479
macro avg	1.00	1.00	1.00	8479
weighted avg	1.00	1.00	1.00	8479

Arduino serial monitor output showing reconstruction error and normal/anomaly detection results

```
Result: Normal activity
Reconstruction error: 0.052647
Result: Normal activity
Reconstruction error: 0.052646
Result: Normal activity
Reconstruction error: 0.052653
Result: Normal activity
Reconstruction error: 0.052656
Result: Normal activity
Reconstruction error: 0.052651
Result: Normal activity
Reconstruction error: 0.052657
Result: Normal activity
Reconstruction error: 0.052707
Result: Normal activity
Reconstruction error: 0.052802
Result: Normal activity
Reconstruction error: 0.052694
Result: Normal activity
```

Section 3: Discussion Questions

Answer the following briefly:

- What threshold did you choose for anomaly detection, and why?

I chose the threshold of 2. I chose this due to the fact that all normal values were below it. There was no overlap between the normal and anomaly (obviously) so it was easy to choose this.

- How does reconstruction error help distinguish normal and anomalous activity?

The reconstruction error helps distinguish between the activities due to the fact that the model is trained to reconstruct normally, so a large error means that the reconstruction is not normal, therefore anomaly. If the model knows how to build normal distributions, then a sample that does not fit those learned patterns will be reconstructed poorly, resulting in a high reconstruction error. Normal activities have low error since the model can reconstruct them accurately.

- What information from the Python notebook must be preserved when deploying the model to Arduino?

The weights of the model have to be saved to deploy the model to Arduino. The weights are converted to int8 in the python notebook and that is saved into a C file so that the model can be deployed. That model is uploaded to the same folder with an Arduino script so it can be uploaded to the Arduino board.

- Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

It is important for it to use the same preprocessing since the model is trained on those assumptions. If the preprocessing changes, then the data inputs to the model may be classified as anomaly since it does not match the 'normal' data that the model understands.

- What are the main limitations of this anomaly detection method when deployed on a microcontroller?
Putting any model on a microcontroller means that there is very limited memory. Therefore, the model has to be usually quantized or undergo some form of memory reduction. This usually degrades the accuracy. For the anomaly detection method, some anomalies may go undetected since the accuracy is degraded or some normals may be false positives.